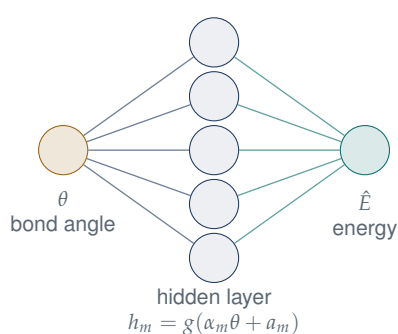


AN UNDERGRADUATE TUTORIAL

Neural Networks for Chemistry Students

*Every equation derived, every constant computed,
every idea tied to something you already know*



A neural model is a function with many adjustable parameters, and a chemist has been fitting such functions since the first calibration curve. The tutorial builds one from that starting point: least squares, then a single nonlinear unit, then a layer, then the chain rule that trains it. Each activation function is shown to be something already familiar, the logistic function as a two-state Boltzmann population and the softplus as a partition function. The running example is the bending potential of water, carried from raw angles to a trained model with every intermediate number printed.

Expanded from the study guide *Neural Networks for Chemistry Students*,
itself based on Dral, Kananenka, Ge and Xue (2023)

How to use this tutorial

Read with a pencil. Every equation that carries a number has that number worked out somewhere in the text, and every number was computed by the script in Appendix A rather than quoted from memory.

Definition or derivation

The mathematics, set out in full. Nothing is asserted that is not derived.

Chemical connection

The place where the mathematics has already appeared in a chemistry course.

Worked example

Arithmetic carried through to the last digit, so that a reader can check each step by hand.

Where students go wrong

Mistakes that produce plausible-looking output and are therefore hard to catch.

Prerequisites

Differentiation including the chain rule and partial derivatives, matrix multiplication, and the Boltzmann distribution. Physical chemistry to the level of a first course in spectroscopy is assumed in the examples, and no programming experience is required to follow the derivations.

Learning outcomes

1. Derive the least-squares solution and explain what a neural model adds to it.
2. Explain why a nonlinear activation function is not optional, with proof.
3. Count the parameters of any feed-forward architecture from its layer sizes.
4. Recognise the logistic, softplus and Gaussian activation functions as objects already met in physical chemistry.
5. Derive the back-propagation equations for a one-hidden-layer model and execute one update by hand.
6. State the stability condition on the learning rate and connect it to step-size control in a geometry optimisation.
7. Diagnose overfitting from a loss curve and name four remedies with their mathematical form.
8. Explain why a potential energy model needs a smooth activation function, in terms of forces.

Notation

$x, \mathbf{x}$	input, scalar or vector	p	number of inputs
y	reference value	$\hat{y}$	model prediction
$w_m, \mathbf{w}$	output-layer weights	b	output-layer bias
α_m	hidden-layer weight	a_m	hidden-layer bias
M	number of hidden neurons	$g(v)$	activation function
h_m	hidden neuron output	z_m	hidden pre-activation, $\alpha_m x + a_m$
θ	all trainable parameters	$L(\theta)$	loss function
γ	learning rate	N	number of training points

Contents

How to use this tutorial	2
Part I. From a calibration curve to a neuron	5
1 The model every chemist has already fitted	5
1.1 Least squares, derived	5
1.2 Why chemistry breaks the straight line	5
2 One neuron	6
3 Layers, and why nonlinearity is compulsory	6
3.1 The collapse theorem	7
3.2 Counting parameters	7
4 Why a hidden layer can represent anything	8
Part II. Activation functions	8
5 The functions and their derivatives	8
6 Every activation function is something you already know	10
Part III. Training	11
7 The loss function	11
8 Gradient descent	11
8.1 How large may the learning rate be?	12
9 Back-propagation, derived in full	12
9.1 Mini-batches, epochs and schedules	14
10 Overfitting	14
10.1 Four remedies, with their mathematics	14
11 Ensembles and honest error bars	16
Part IV. Beyond the feed-forward model	16
12 Convolutional models	16
13 Recurrent models and the vanishing gradient	17
13.1 Why long sequences fail	17
13.2 Gated architectures	17
Part V. The bending potential of water, end to end	18
14 Setting up the problem	18
14.1 Data preparation	18
15 Forward pass, training and results	19
Part VI. Problems	20

A The script	22
B Solutions	26

Part I. From a calibration curve to a neuron

1 The model every chemist has already fitted

A neural model is often introduced as something new. It is not. Its ancestor is the straight line drawn through a calibration series in the first year of a chemistry degree, and the whole subject is best understood as a sequence of modifications to that line.

1.1 Least squares, derived

Suppose N measurements (x_i, y_i) and the model $\hat{y} = wx + b$. The loss is the sum of squared residuals,

$$L(w, b) = \sum_{i=1}^N (wx_i + b - y_i)^2. \quad (1)$$

Minimise by setting both partial derivatives to zero. Differentiating with respect to b ,

$$\frac{\partial L}{\partial b} = 2 \sum_i (wx_i + b - y_i) = 0 \implies b = \bar{y} - w\bar{x}, \quad (2)$$

where $\bar{x}$ and $\bar{y}$ are the sample means. Differentiating with respect to w and substituting for b ,

$$\frac{\partial L}{\partial w} = 2 \sum_i x_i (wx_i + b - y_i) = 0 \implies w = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}. \quad (3)$$

Two features of the result matter for everything that follows. The solution is *closed form*, obtained in one step with no iteration. And it exists because L is a quadratic function of the parameters, so the stationary point is unique and is a minimum.

The general linear model in matrix form

With p inputs, collect them in $\mathbf{X} \in \mathbb{R}^{N \times p}$ with a column of ones appended for the intercept. The prediction is $\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\beta}$ and

$$L = \|\mathbf{X}\boldsymbol{\beta} - \mathbf{y}\|^2, \quad \frac{\partial L}{\partial \boldsymbol{\beta}} = 2\mathbf{X}^\top (\mathbf{X}\boldsymbol{\beta} - \mathbf{y}) = \mathbf{0}, \quad (4)$$

giving the *normal equations* and their solution

$$\boldsymbol{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (5)$$

Chemical connection: the calibration curve

Equation (3) is the slope of a Beer–Lambert plot. With absorbance A on the vertical axis and concentration c on the horizontal, the fitted slope is $\epsilon\ell$ and the intercept is the blank. Reporting a molar absorption coefficient is therefore reporting the single trained parameter of a linear model. An analytical chemist who has propagated the uncertainty on that slope has already performed the error analysis of a machine-learned model.

1.2 Why chemistry breaks the straight line

A linear model asserts that the effect of changing an input is the same wherever you are. Chemistry rarely obliges.

Relationship	Actual form	Consequence
energy against bond length	Morse, $D_e(1 - e^{-a(r-r_e)})^2$	steeply repulsive inside r_e , flat outside
rate against temperature	Arrhenius, $Ae^{-E_a/RT}$	linear only in $\ln k$ against $1/T$
pH against added titrant	sigmoidal	almost flat, then almost vertical
energy against bond angle	anharmonic well	curvature changes sign before the linear geometry

A chemist's traditional response is to linearise by transforming the axes, which is what an Arrhenius plot does. The strategy works when the functional form is known in advance. When it is not, and in most property-prediction problems it is not, a model that can learn the shape is needed.

2 One neuron

A single neuron

A neuron takes a linear combination of its inputs, adds a bias, and passes the result through a fixed nonlinear function:

$$h(x) = g(\underbrace{\alpha x + a}_z), \quad (6)$$

where α is the *weight*, a the *bias*, z the *pre-activation* and g the *activation function*. With p inputs, $z = \alpha^\top x + a$.

The two parameters have clean geometric meanings, and students who grasp them stop finding neural models mysterious.

- *The weight sets the steepness.* Doubling α compresses the horizontal axis by a factor of two, so the transition in g becomes twice as sharp.
- *The bias sets the position.* The activation turns on where $z = 0$, that is at $x = -a/\alpha$. Changing a slides the feature along the axis without changing its shape.

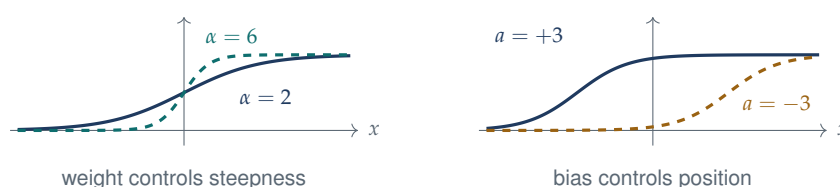


Figure 1: The two parameters of a neuron, illustrated with the logistic activation function. Every feature a hidden layer builds is assembled from steepness and position, which is why a large number of neurons is needed to represent a complicated curve.

3 Layers, and why nonlinearity is compulsory

Place M neurons side by side, all reading the same input, and combine their outputs linearly:

$$\hat{y} = b + \sum_{m=1}^M w_m h_m = b + \sum_{m=1}^M w_m g(\alpha_m^\top x + a_m). \quad (7)$$

Equation (7) is a complete feed-forward model with one hidden layer. Reading it from the inside out gives the layer picture: an *input layer* holding x , a *hidden layer* holding h , and an *output layer* producing $\hat{y}$. The architecture is written $p : M : 1$.

Composition of layers

Writing $f^{(1)}(\mathbf{x}) = \mathbf{h}$ and $f^{(2)}(\mathbf{h}) = \hat{\mathbf{y}}$,

$$\hat{\mathbf{y}} = f^{(2)}(f^{(1)}(\mathbf{x})), \quad (8)$$

and with N_L layers, $\hat{\mathbf{y}} = f^{(N_L)} \circ \dots \circ f^{(1)}(\mathbf{x})$. Depth means repeated composition, which is the reason the chain rule governs training.

3.1 The collapse theorem**Proposition.**

If every activation function is the identity, $g(v) = v$, then a model of any depth and any width computes an affine function of its inputs, and is therefore exactly equivalent to multiple linear regression.

Proof. With g the identity, layer l computes $\mathbf{h}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}$. Composing two such layers,

$$\begin{aligned} \mathbf{h}^{(2)} &= \mathbf{W}^{(2)}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} \\ &= \underbrace{(\mathbf{W}^{(2)}\mathbf{W}^{(1)})}_{\mathbf{W}'}\mathbf{x} + \underbrace{(\mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)})}_{\mathbf{b}'}, \end{aligned} \quad (9)$$

which has the form of a single affine layer. Induction on the number of layers completes the argument. ■

The proposition is not a technicality. It says that all the representational power of a neural model comes from g , and that a model with a linear activation function has as many parameters as before and exactly as much expressive power as a straight line. Section 14 shows the failure happening on real data.

3.2 Counting parameters**Parameter count**

For a fully connected model with layer sizes $N_1, N_2, \dots, N_L$, layer l carries a weight matrix of size $N_{l-1} \times N_l$ and a bias vector of length N_l , so

$$N_{\text{params}} = \sum_{l=2}^L (N_{l-1} + 1)N_l. \quad (10)$$

For $p : M : 1$ this gives $(p + 1)M + (M + 1) = pM + 2M + 1$.

Worked example: how fast the count grows

Architecture	Parameters	Comment
1 : 1 : 1 (identity g)	4	equivalent to a straight line, which needs 2
1 : 32 : 1	97	the model used in Section 14
1 : 64 : 1	193	one input, still nearly 200 parameters
3 : 64 : 64	4416	a single hidden layer of a small property model
384 : 128 : 1	49409	a descriptor-based energy model

Compare with the two parameters of the calibration line in Section 1. Published models for quantum chemistry go much further: a three-hidden-layer architecture with over 300 000 parameters, used as an ensemble of eight, exceeds 2.5 million.

Where students go wrong: forgetting the biases

The count $(N_{l-1} + 1)N_l$ has a +1 for the bias of each neuron in layer l , not one bias for the layer. A 1 : 32 : 1 model has 32 hidden biases and one output bias, so the total is $32 + 32 + 32 + 1 = 97$ and not $32 + 1 + 1$. Miscounting here leads students to underestimate model size by a factor of two.

4 Why a hidden layer can represent anything

The universal approximation theorem states that a model of the form (7), with a suitable non-polynomial g and enough hidden neurons, can approximate any continuous function on a bounded interval to any desired accuracy. The formal proof is analytical, but the mechanism is constructive and can be seen in two lines.

Worked example: building a bump from two sigmoids

Take the logistic function $\sigma(v) = 1/(1 + e^{-v})$ and form

$$B(x) = \sigma(k(x+1)) - \sigma(k(x-1)), \quad (11)$$

which is a two-neuron model with $w_1 = +1$, $w_2 = -1$. With $k = 10$,

x	-2	-1	0	+1	+2
$B(x)$	0.00005	0.50000	0.99991	0.50000	0.00005

The result is a rectangular pulse of unit height, centred at the origin, with width set by the biases and edge sharpness set by k . Two neurons make one bump. A weighted sum of many such bumps is a histogram approximation of any target curve, and refining the bumps refines the approximation without limit.

Chemical connection: a basis set expansion

The construction is a basis set expansion. Each hidden neuron is one basis function, the output weights w_m are the expansion coefficients, and the readout $\hat{y} = b + \sum_m w_m h_m$ is the expansion itself. The essential difference from a Gaussian basis in electronic structure is that the exponents and centres of the basis functions, here α_m and a_m , are optimised along with the coefficients rather than fixed in advance. A neural model is a fully variational basis set.

Where students go wrong: reading too much into universality

The theorem says a representation *exists*. It does not say that gradient descent will find it, that the number of neurons required is modest, or that the result will generalise beyond the training data. In practice all three are the real difficulties, and the theorem should be treated as reassurance that nothing is fundamentally impossible rather than as a guarantee of success.

Part II. Activation functions

5 The functions and their derivatives

Since training uses gradients, every activation function must be considered together with its derivative. The derivatives are collected here and derived below.

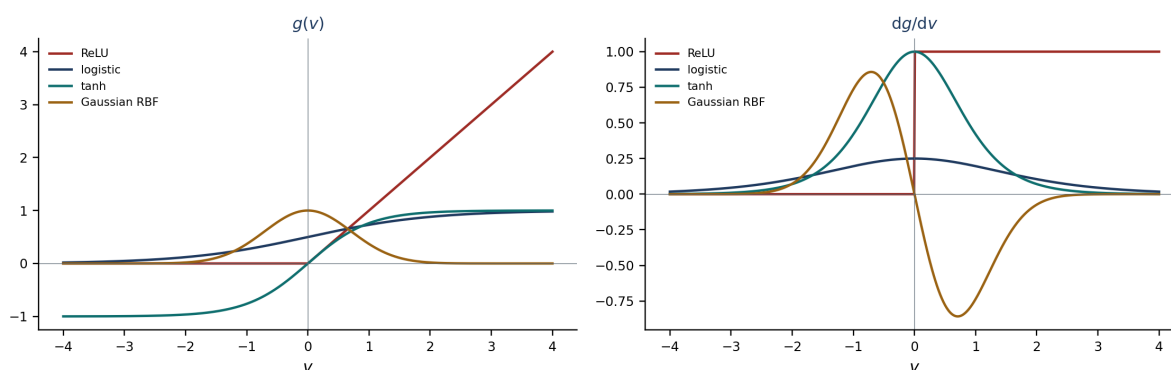


Figure 2: Four activation functions and their derivatives. Note the step in the ReLU derivative at $v = 0$. Any model built from ReLU units has a potential energy surface whose forces jump discontinuously across the surfaces where a unit switches on, which is the reason smooth activation functions are preferred in chemistry.

Name	$g(v)$	$g'(v)$	Range	Smooth?
identity	v	1	$(-\infty, \infty)$	yes, and useless
ReLU	$\max(0, v)$	0 or 1	$[0, \infty)$	no, kink at 0
logistic	$\sigma(v) = \frac{1}{1 + e^{-v}}$	$\sigma(1 - \sigma)$	$(0, 1)$	yes
tanh	$\tanh v$	$1 - \tanh^2 v$	$(-1, 1)$	yes
softplus	$\ln(1 + e^v)$	$\sigma(v)$	$(0, \infty)$	yes
Gaussian (RBF)	$e^{-a(v-c)^2}$	$-2a(v-c)e^{-a(v-c)^2}$	$(0, 1]$	yes
GELU	$v \frac{1}{2} [1 + \operatorname{erf}(v/\sqrt{2})]$	$\frac{1}{2} [1 + \operatorname{erf}] + \frac{ve^{-v^2/2}}{\sqrt{2\pi}}$	$\approx (-0.17, \infty)$	yes

Derivation of the three standard derivatives

Logistic. Write $\sigma = (1 + e^{-v})^{-1}$. By the chain rule,

$$\sigma' = -(1 + e^{-v})^{-2} \cdot (-e^{-v}) = \frac{e^{-v}}{(1 + e^{-v})^2} = \frac{1}{1 + e^{-v}} \cdot \frac{e^{-v}}{1 + e^{-v}} = \sigma(1 - \sigma),$$

using $\frac{e^{-v}}{1 + e^{-v}} = 1 - \sigma$. The maximum slope is at $v = 0$, where $\sigma = \frac{1}{2}$ and $\sigma' = \frac{1}{4}$.

Hyperbolic tangent. From $\tanh v = (e^v - e^{-v}) / (e^v + e^{-v})$ and the quotient rule,

$$\frac{d}{dv} \tanh v = \frac{(e^v + e^{-v})^2 - (e^v - e^{-v})^2}{(e^v + e^{-v})^2} = 1 - \tanh^2 v,$$

with maximum slope 1 at the origin, four times that of the logistic function.

Softplus. $\frac{d}{dv} \ln(1 + e^v) = \frac{e^v}{1 + e^v} = \frac{1}{1 + e^{-v}} = \sigma(v)$. The softplus function is the integral of the logistic function, a relation used in the next section.

6 Every activation function is something you already know

The logistic function is a two-state Boltzmann population

Consider two states separated by ΔE , with the lower state taken as the zero of energy. The population of the upper state is

$$p_{\text{upper}} = \frac{e^{-\Delta E/k_{\text{B}}T}}{1 + e^{-\Delta E/k_{\text{B}}T}} = \frac{1}{1 + e^{+\Delta E/k_{\text{B}}T}} = \sigma\left(-\frac{\Delta E}{k_{\text{B}}T}\right). \quad (12)$$

The logistic activation function *is* the two-state Boltzmann population, with the pre-activation v playing the role of $-\Delta E/k_{\text{B}}T$. The weight α is an inverse temperature: a large weight is a sharp, low-temperature switch, and a small weight is a soft, high-temperature one. At 300 K, $k_{\text{B}}T = 208.5 \text{ cm}^{-1}$, and

$\Delta E / \text{cm}^{-1}$	0	200	600	1500
p_{upper}	0.5000	0.2770	0.0533	0.0008

The same numbers appear in a spectroscopy problem set and in the forward pass of a neural model.

The softplus function is a partition function

For the same two-state system with the upper state at $-vk_{\text{B}}T$ relative to the lower,

$$Z = 1 + e^v, \quad \ln Z = \ln(1 + e^v) = \text{softplus}(v). \quad (13)$$

The identity $\text{softplus}'(v) = \sigma(v)$ derived above is then the standard thermodynamic statement that differentiating $\ln Z$ with respect to the reduced energy returns the population. A student who has derived $\langle E \rangle = -\partial \ln Z / \partial \beta$ has derived the softplus gradient.

The Gaussian activation function is a basis function

$g(v) = e^{-a(v-c)^2}$ is a normalised Gaussian without its prefactor, centred at c with width $a^{-1/2}$. The hidden layer of a Gaussian model is a set of localised bumps placed along the input axis, exactly like a set of primitive Gaussians placed along a bond coordinate. Each neuron responds only near its own centre, so the model is a sum of local contributions rather than a sum of global ones. Locality is why Gaussian units fit potential energy curves so efficiently and why they extrapolate badly: outside the range covered by the centres, every neuron returns nearly zero and the prediction decays to the bias b .

Why ReLU is a poor choice for a potential energy surface

The force on a nucleus is $F = -\partial E / \partial q$. If E is represented by a model with ReLU units then $\partial E / \partial q$ contains the step function g' , so the predicted force jumps discontinuously whenever a unit crosses its threshold. A molecular dynamics trajectory on such a surface does not conserve energy, and a geometry optimiser that uses the gradient will oscillate near the discontinuities. Smooth alternatives, notably GELU and the Gaussian, remove the problem at negligible cost. The requirement is stronger than mere differentiability: for a vibrational analysis the second derivative is needed as well, so the activation function should be at least twice continuously differentiable.

Where students go wrong: vanishing gradients

From the table, $\sigma' \leq \frac{1}{4}$ everywhere. In a model with n stacked logistic layers, the chain rule multiplies n such factors, so the gradient reaching the first layer is smaller by at most

4^{-n} . For $n = 10$ that is 10^{-6} , and the early layers effectively stop learning. Understanding the bound explains why tanh, whose derivative reaches 1, was preferred for years, and why ReLU-like functions with unit derivative over half their range displaced both in deep architectures.

Part III. Training

7 The loss function

Sum of squared errors

$$L(\theta) = \sum_{i=1}^N [f(x_i; \theta) - y_i]^2. \quad (14)$$

When several properties are learned at once, the total loss is a weighted sum, $L = \sum_k c_k L_k$, with the weights c_k chosen to balance the units and the relative importance of the properties.

The squared form is not arbitrary. If the reference values carry independent Gaussian errors of equal variance, then the probability of observing the data given the parameters is

$$P(y | \theta) \propto \prod_i \exp \left[-\frac{(f_i - y_i)^2}{2s^2} \right], \quad -\ln P = \frac{1}{2s^2} \sum_i (f_i - y_i)^2 + \text{const.} \quad (15)$$

Minimising the sum of squares is therefore maximising the likelihood. The assumption matters: if some reference energies are far less reliable than others, weighting each term by $1/s_i^2$ is the correct treatment and plain least squares is not.

Chemical connection: weighting energies against forces

A potential energy model is usually trained on both energies and forces, with $L = L_E + c L_F$. Energies come in cm^{-1} or hartree and forces in hartree per bohr, so c carries units and its value decides which property the model prefers. Setting c too small produces accurate energies with useless gradients, which is the most common failure in a first attempt at a machine-learned potential.

8 Gradient descent

Unlike Equation (5), the loss of a nonlinear model has no closed-form minimiser. The remedy is iteration.

The update rule, from a Taylor expansion

Expand the loss about the current parameters $\theta^{(r)}$ for a small step $\Delta\theta$:

$$L(\theta^{(r)} + \Delta\theta) \approx L(\theta^{(r)}) + \nabla L^\top \Delta\theta. \quad (16)$$

For a step of fixed length, the first-order decrease is largest when $\Delta\theta$ points along $-\nabla L$, which gives

$$\theta_k^{(r+1)} = \theta_k^{(r)} - \gamma \frac{\partial L}{\partial \theta_k}. \quad (17)$$

The scalar γ is the *learning rate*.

8.1 How large may the learning rate be?

Apply (17) to the one-dimensional quadratic $L = \frac{1}{2}k\theta^2$, for which $\partial L/\partial\theta = k\theta$. The update is

$$\theta^{(r+1)} = \theta^{(r)} - \gamma k \theta^{(r)} = (1 - \gamma k) \theta^{(r)}, \quad (18)$$

so after r steps $\theta^{(r)} = (1 - \gamma k)^r \theta^{(0)}$. Convergence requires $|1 - \gamma k| < 1$, that is

$$0 < \gamma < \frac{2}{k}. \quad (19)$$

In several dimensions k is replaced by the largest eigenvalue of the Hessian, so the permissible step is dictated by the stiffest direction while the slowest direction sets how many steps are needed. The ratio of the two, the condition number, is what makes optimisation hard.

Worked example: the stability threshold in action

With $k = 10$, Equation (19) permits $\gamma < 0.2$. Starting from $\theta^{(0)} = 1$ and taking 30 steps:

γ	$\theta^{(30)}$	behaviour
0.05	9.3×10^{-10}	rapid, stable convergence
0.19	4.2×10^{-2}	convergent but oscillating, just inside the bound
0.21	17.5	divergent, outside the bound

The transition is sharp and sits exactly at the predicted $\gamma = 0.2$.

Chemical connection: this is a geometry optimisation

Training and geometry optimisation are the same procedure applied to different surfaces. In a geometry optimisation the coordinates are nuclear positions, the objective is the electronic energy, the gradient is the force, and the step size must respect the curvature of the stiffest vibrational mode. In training the coordinates are weights, the objective is the loss, and the same stability condition applies. Momentum is the analogue of damped dynamics, and the adaptive methods used in practice are cousins of the quasi-Newton schemes that a computational chemist already relies on. A student who understands why an optimiser fails on a floppy torsion understands why training fails on an ill-conditioned loss.

9 Back-propagation, derived in full

Consider the 1 : M : 1 model

$$z_m = \alpha_m x + a_m, \quad h_m = g(z_m), \quad \hat{y} = b + \sum_m w_m h_m, \quad (20)$$

with the single-point loss $L = (\hat{y} - y)^2$ and $e \equiv \hat{y} - y$. All four gradients follow from the chain rule.

The four gradients

$$\frac{\partial L}{\partial b} = 2e, \quad (21)$$

$$\frac{\partial L}{\partial w_m} = 2e h_m, \quad (22)$$

$$\frac{\partial L}{\partial a_m} = \underbrace{2e w_m g'(z_m)}_{\delta_m}, \quad (23)$$

$$\frac{\partial L}{\partial \alpha_m} = \delta_m x. \quad (24)$$

Derivation. Since $\partial \hat{y} / \partial b = 1$ and $\partial L / \partial \hat{y} = 2e$, Equation (21) is immediate. For w_m , $\partial \hat{y} / \partial w_m = h_m$, giving (22). For the hidden parameters the chain must pass through three links,

$$\frac{\partial L}{\partial a_m} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_m} \cdot \frac{\partial h_m}{\partial z_m} \cdot \frac{\partial z_m}{\partial a_m} = 2e \cdot w_m \cdot g'(z_m) \cdot 1, \quad (25)$$

and the only difference for α_m is the last factor, $\partial z_m / \partial \alpha_m = x$. ■

Three structural facts are worth naming, because they generalise to models of any depth.

1. **The error is shared.** Every gradient contains the same factor $2e$. One error signal is computed at the output and distributed backwards.
2. **Weights route the error.** The hidden gradient carries w_m , so a neuron with a small output weight receives a correspondingly small correction.
3. **The activation gates the flow.** The factor $g'(z_m)$ closes the channel wherever the activation function is flat, which is the mechanism behind the vanishing gradient.

The general recursion

For a model with layers $l = 1, \dots, L$, pre-activations $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}$ and $\mathbf{h}^{(l)} = g(\mathbf{z}^{(l)})$, define $\delta^{(l)} = \partial L / \partial \mathbf{z}^{(l)}$. Then

$$\delta^{(L)} = \nabla_{\hat{\mathbf{y}}} L \odot g'(\mathbf{z}^{(L)}), \quad (26)$$

$$\delta^{(l)} = (\mathbf{W}^{(l+1)\top} \delta^{(l+1)}) \odot g'(\mathbf{z}^{(l)}), \quad (27)$$

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} \mathbf{h}^{(l-1)\top}, \quad \frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}, \quad (28)$$

where $\odot$ denotes elementwise multiplication. The recursion is back-propagation, and its cost is the same as one forward pass.

Worked example: one complete update by hand

A 1 : 2 : 1 model with tanh activation, at the normalised training point corresponding to $\theta = 100^\circ$, so $x = -0.9721$ and $y = -1.2401$. Take $\alpha = (1.5, -2.0)$, $a = (0.5, 0.4)$, $w = (0.8, -0.6)$, $b = 0.1$.

Forward pass.

$$z_1 = 1.5(-0.9721) + 0.5 = -0.9582, \quad z_2 = -2.0(-0.9721) + 0.4 = 2.3443,$$

$$h_1 = \tanh(-0.9582) = -0.7435, \quad h_2 = \tanh(2.3443) = 0.9818,$$

$$\hat{y} = 0.1 + 0.8(-0.7435) + (-0.6)(0.9818) = -1.0838.$$

The error is $e = \hat{y} - y = -1.0838 - (-1.2401) = +0.1563$ and $L = e^2 = 0.02443$.

Backward pass. From (21) and (22),

$$\frac{\partial L}{\partial b} = 2(0.1563) = 0.3126, \quad \frac{\partial L}{\partial w} = 2e \mathbf{h} = (-0.2324, 0.3069).$$

With $g'(z) = 1 - \tanh^2 z$, so $g'(z_1) = 1 - 0.7435^2 = 0.4472$ and $g'(z_2) = 1 - 0.9818^2 = 0.0360$,

$$\delta = 2e \mathbf{w} \odot g'(\mathbf{z}) = (0.1118, -0.0068), \quad \frac{\partial L}{\partial \alpha} = \delta \mathbf{x} = (-0.1087, 0.0066).$$

Update and check. With $\gamma = 0.1$, the output weights become $(0.8232, -0.6307)$ and the bias 0.0687 . Recomputing the forward pass with all updated parameters gives $\hat{y} = -1.1704$ and $L = 0.00486$, down from 0.02443 . One step has removed 80% of the loss at this point.

Read the numbers. Neuron 2 is saturated, $h_2 = 0.9818$, so $g'(z_2) = 0.036$ and its incoming gradient is smaller than neuron 1's by a factor of twelve. Saturation is not a hypothetical concern, it is visible in a two-neuron model on the first step.

9.1 Mini-batches, epochs and schedules

Computing ∇L over the whole training set is exact and expensive. Splitting the set into *mini-batches* and taking one step per batch gives a noisy but unbiased estimate of the gradient, since the expectation over a randomly drawn batch equals the full gradient. One sweep through all the data is an *epoch*. The noise is not purely a cost, since it helps the optimiser leave shallow minima, in the same way that a finite temperature helps a system escape a local structure.

A learning rate schedule starts large, to cross the landscape, and decreases, to settle into a minimum. The strategy is simulated annealing under another name.

10 Overfitting

A model with 97 parameters fitted to 16 points can pass exactly through every point. Doing so is not success.

Overfitting and underfitting

Overfitting is the regime in which the training loss continues to fall while the validation loss rises, because the model is reproducing the noise of the training set. **Underfitting** is the opposite, in which both losses remain high because the model cannot represent the target at all.

10.1 Four remedies, with their mathematics

Early stopping. Evaluate the validation loss each epoch, save the parameters whenever it reaches a new minimum, and stop after a fixed number of epochs without improvement, called the *patience*. In Figure 3 the correct stopping point is epoch 137, at which the validation loss is 1.3×10^{-4} against 4.9×10^{-4} at epoch 3000. Early stopping costs nothing and should be the default.

Regularisation. Add a penalty on the size of the parameters,

$$L(\theta) = \sum_i [f(x_i; \theta) - y_i]^2 + \lambda \sum_j \theta_j^2. \quad (29)$$

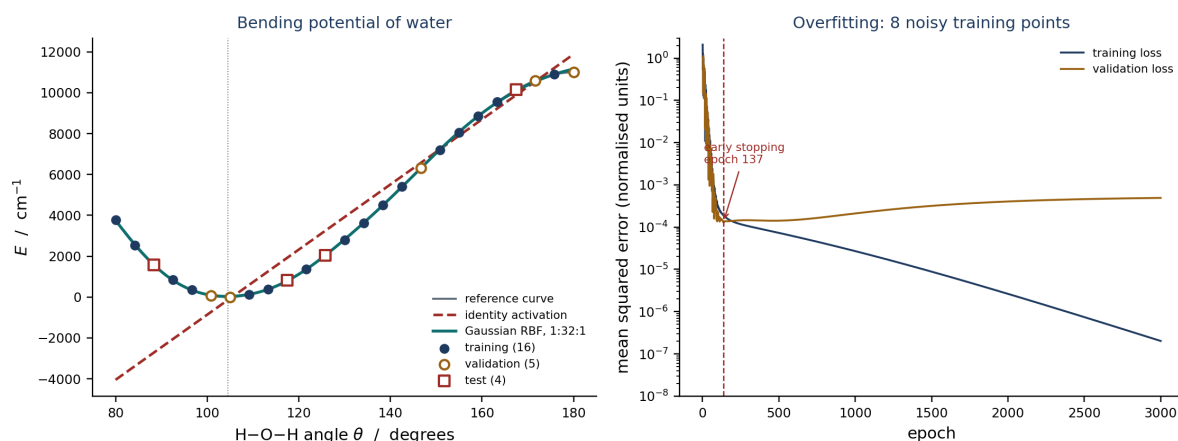


Figure 3: *Left:* the bending potential of water, fitted by two 1 : 32 : 1 models differing only in the activation function. The identity activation produces the straight dashed line, exactly as the collapse theorem of Section 3 requires. The Gaussian model reproduces the curve to 20 cm^{-1} on the training points. *Right:* the same architecture trained on 8 deliberately noisy points, showing the classic signature of overfitting. The validation loss reaches its minimum at epoch 137 and rises thereafter while the training loss keeps falling.

For the linear model the effect can be computed exactly. Repeating the derivation of (5) with the extra term,

$$\frac{\partial L}{\partial \beta} = 2X^T(X\beta - y) + 2\lambda\beta = 0 \implies \beta = (X^T X + \lambda I)^{-1} X^T y. \quad (30)$$

Adding λ to every eigenvalue of $X^T X$ lowers the condition number and shrinks the fitted coefficients, which is why the method is also called ridge regression. In the Bayesian reading, λ is the precision of a Gaussian prior that says large parameters are unlikely.

Dropout. During training, set a random fraction of hidden outputs to zero at each step. No neuron can be relied upon, so the model is forced to spread the representation across many units. The procedure approximates an average over an exponentially large family of thinned models at the cost of one.

Data augmentation. Add training points that are free, in the sense that they require no new quantum chemistry. The most useful case in chemistry is permutation of identical atoms, which leaves the energy unchanged but changes the input vector whenever the representation depends on atom ordering.

Worked example: how many free training points?

The number of relabellings of identical atoms is the product of the factorials of the counts of each element:

Molecule	Equivalent labellings	
H ₂ O	1! 2! = 2	modest
CH ₄	1! 4! = 24	one calculation becomes 24 training points
C ₂ H ₆ O	2! 6! 1! = 1440	
C ₆ H ₆	6! 6! = 518 400	sample, do not enumerate

For benzene the full set is far too large to use, so a random sample of permutations is drawn each epoch. Note that a model built to be permutation invariant by construction needs none of this, which is one argument for such architectures.

11 Ensembles and honest error bars

Training is not deterministic. Different random initialisations converge to different parameter sets, all of which fit the training data. Rather than suppressing the variability, use it.

Train K models independently and report the mean prediction $\bar{f} = \frac{1}{K} \sum_k f_k$. If the individual errors were independent with variance s^2 , the variance of the mean would be s^2/K , so an ensemble of eight halves the random error. More importantly, the spread among members estimates the uncertainty of the prediction.

Worked example: reading an ensemble

Eight models predict the energy of a molecule, in hartree:

−76.4312, −76.4298, −76.4325, −76.4301, −76.4288, −76.4319, −76.4307, −76.4295.

The mean is −76.43056 E_h , the sample standard deviation is $1.25 \times 10^{-3} E_h$, and the standard error of the mean is $4.43 \times 10^{-4} E_h$. Converting the spread to chemical units gives 3.29 kJ mol^{−1}, which is comparable to the accuracy being sought and is therefore a warning rather than a reassurance. A large spread means the models disagree, which usually means the input lies outside the region the training set covered.

Where students go wrong: confusing spread with accuracy

The ensemble spread measures how much the prediction depends on the training procedure. It does not measure the error against experiment or against the underlying quantum chemical method. A tightly clustered ensemble trained on systematically biased reference data will be confidently wrong, and no amount of ensembling detects the bias.

Part IV. Beyond the feed-forward model

12 Convolutional models

A fully connected layer reading n inputs and producing n outputs needs $n^2 + n$ parameters and treats every input position as unrelated to every other. For data laid out on a grid, such as a spectrum sampled at equal wavenumber intervals or a trajectory sampled at equal times, both properties are wasteful.

Discrete convolution

For an input sequence x and a kernel w of length k ,

$$(x * w)_i = \sum_{j=1}^k x_{i+j-1} w_{k+1-j}. \quad (31)$$

The output at position i depends only on k neighbouring inputs, and the same k weights are used at every position.

Two properties follow, and they are the whole reason such layers are used. *Sparse connectivity* means each output touches only a local window. *Parameter sharing* means the kernel is reused, so the count is $k + 1$ regardless of the length of the input. A layer over a 2000-point spectrum with a kernel of length 9 has 10 parameters, against 4 002 000 for the fully connected equivalent.

Chemical connection: Savitzky–Golay smoothing is a convolution

A chemist smoothing a noisy spectrum with a Savitzky–Golay filter is applying Equation (31) with a fixed kernel derived from a local polynomial fit. Taking the derivative of a spectrum is the same operation with a different fixed kernel. A convolutional layer is exactly that operation with the kernel *learned* from data rather than derived analytically. The translation-invariance argument is also familiar: a peak means the same thing whether it sits at 1600 or 1700 cm^{-1} , so the detector for it should not depend on position.

13 Recurrent models and the vanishing gradient

For sequential data the natural structure is recursion. A hidden state is carried forward and updated at each step,

$$h_t = R(h_{t-1}, x_t; \theta), \quad \text{concretely} \quad h_t = \tanh(b + W_h h_{t-1} + W_x x_t). \quad (32)$$

The state h_t is a compressed memory of everything seen up to step t , and the same parameters act at every step, which is the temporal analogue of the parameter sharing in Section 12.

13.1 Why long sequences fail

Training requires the sensitivity of a late state to an early one. Applying the chain rule along the recursion,

$$\frac{\partial h_T}{\partial h_1} = \prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} = \prod_{t=2}^T \text{diag}(g'(z_t)) W_h. \quad (33)$$

Equation (33) is a product of $T - 1$ matrices. If the typical magnitude of each factor is ρ , then the product scales as ρ^{T-1} , which decays to zero for $\rho < 1$ and explodes for $\rho > 1$. Since $|\tanh'| \leq 1$, decay is the usual outcome.

Worked example: the size of the problem

Situation	Factor	Consequence
$\rho = 0.5, T = 20$	9.5×10^{-7}	no learning of long-range structure
$\rho = 0.7, T = 30$	2.3×10^{-5}	effectively the same
$\rho = 0.9, T = 20$	1.2×10^{-1}	usable
$\rho = 1.1, T = 20$	6.7	gradients explode, training destabilises

The window of usable ρ is narrow and cannot be maintained by tuning alone, which motivates an architecture that carries information without repeated multiplication.

13.2 Gated architectures

The long short-term memory unit adds a *cell state* C_t that is modified additively rather than multiplicatively, together with three learned gates that decide what to discard, admit and expose:

$$C_t = F_t \odot C_{t-1} + I_t \odot G_t, \quad (34)$$

$$h_t = O_t \odot \tanh(C_t), \quad (35)$$

where F , I and O are the forget, input and output gates, each a logistic function of h_{t-1} and x_t , and G is the candidate update. When the forget gate is near one the cell state passes through unchanged, so the derivative $\partial C_t / \partial C_{t-1} \approx 1$ and the product in (33) no longer decays. The gated recurrent unit merges the forget and input gates into one update gate and dispenses with a separate cell state.

Parameter counts

Let l be the hidden state length and n the input length. A recurrent layer holds $W_h \in \mathbb{R}^{l \times l}$, $W_x \in \mathbb{R}^{l \times n}$ and a bias of length l , so

$$N_{\text{RNN}} = l(l + n + 1). \quad (36)$$

The gated units repeat the same block for each gate, giving $N_{\text{LSTM}} = 4l(l + n + 1)$ and $N_{\text{GRU}} = 3l(l + n + 1)$.

Worked example: a two-layer model of three excited-state populations

With $n = 3$ inputs and hidden length $l = 25$:

	RNN	GRU	LSTM
layer 1, $l(l + n + 1) = 25(25 + 3 + 1)$	725	2 175	2 900
layer 2, $l(l + l + 1) = 25(25 + 25 + 1)$	1 275	3 825	5 100
total	2 000	6 000	8 000

The gated models cost three and four times the plain recurrent one, which is the price of solving the gradient problem. For the short sequences typical of population dynamics in nonadiabatic simulations, the plain model is often sufficient, and the choice should be tested rather than assumed.

Part V. The bending potential of water, end to end

14 Setting up the problem

The target is the electronic energy of H_2O as a function of the H–O–H angle θ , with both O–H distances held at 0.9584 Å. The input is one number, so $p = 1$, and the output is one number.

About the reference data.

So that every number here can be reproduced without a quantum chemistry package, the reference curve is an analytic model bending potential, $E(u) = \frac{1}{2}ku^2 + c_3u^3 + c_4u^4$ with $u = \theta - \theta_e$ in radians. The two cubic and quartic coefficients are fixed by requiring the linear geometry to lie 11 000 cm^{-1} above the minimum, the experimental barrier to linearity of water, and to be a stationary point by symmetry. With $k = 35\,239 \text{ cm}^{-1} \text{ rad}^{-2}$ from a bending force constant of 0.70 mdyn Å rad^{-2} , this gives $c_3 = -7512$ and $c_4 = -798 \text{ cm}^{-1}$ per radian to the appropriate power. The curve behaves like a real bending potential and stands in for a CCSD(T) scan.

θ / degrees	80	90	104.5	120	150	180
E / cm^{-1}	3782	1247	0	1136	7032	11 000

Note that the curve is not symmetric about the minimum: compressing by 24.5° costs 3782 cm^{-1} , whereas opening by the same amount costs about 1100 cm^{-1} . Bending potentials are anharmonic, and a straight line cannot begin to represent the shape.

14.1 Data preparation

Twenty-five angles are computed between 80° and 180° and split at random into 16 training, 5 validation and 4 test points. The split is drawn once and fixed.

Input normalisation

$$\tilde{x} = \frac{\theta - \mu_\theta}{\sigma_\theta}, \quad (37)$$

with μ_θ and σ_θ computed from the *training set only*. For this split, $\mu_\theta = 127.92^\circ$ and $\sigma_\theta = 28.72^\circ$, so $\theta = 100^\circ$ becomes $\tilde{x} = -0.9721$. The targets are normalised the same way and the prediction is converted back afterwards.

Where students go wrong: normalising with the wrong statistics

Computing μ and σ over the whole dataset, including validation and test points, leaks information from the held-out data into training. The effect is small for a mean and standard deviation and can be large for other preprocessing choices, and the habit of computing every statistic on the training set alone is the one worth forming.

Normalisation matters because of Section 8. Raw angles have magnitudes around 10^2 and raw energies around 10^4 , so the curvature of the loss differs between parameter directions by many orders of magnitude and no single learning rate serves them all. After normalisation the curvatures are comparable and a single γ works.

15 Forward pass, training and results

The architecture is 1 : 32 : 1 with Gaussian hidden units and an identity output, so by Equation (10) the model has 97 parameters. The output layer is left linear because the energy is an unrestricted real number, and imposing a bounded activation there would cap the predictable range.

Worked example: the forward pass at $\theta = 100^\circ$

The normalised input is $\tilde{x} = -0.9721$. Hidden neuron m computes

$$h_m = g(\alpha_m \tilde{x} + a_m) = \exp[-(\alpha_m \tilde{x} + a_m)^2].$$

Taking $\alpha_1 = 2.5$ and $a_1 = -0.3$ as an illustration,

$$z_1 = 2.5(-0.9721) - 0.3 = -2.7303, \quad h_1 = e^{-2.7303^2} = e^{-7.4545} = 5.8 \times 10^{-4}.$$

The neuron is essentially silent at this input, because its centre lies far from $\tilde{x}$. Repeating for all 32 neurons gives the vector $\mathbf{h}$, and the output is $\hat{E} = b + \mathbf{h}^\top \mathbf{w}$, converted back to cm^{-1} by inverting the target normalisation. The locality of the Gaussian units means only a handful of neurons contribute at any one angle, which is the sense in which such a model is a set of local basis functions.

Training used full-batch gradient descent with momentum, learning rate 0.05, for 4000 epochs, with the parameters saved at the lowest validation loss. The whole procedure is 120 lines of NumPy with the gradients of Section 9 written out by hand.

Activation	Train RMSE	Validation RMSE	Test RMSE
	cm^{-1}		
identity	2667	549	2301
Gaussian (RBF)	19.8	78.8	33.4

The Gaussian model reproduces the reference curve to better than 100 cm^{-1} everywhere and places the minimum at 104.35° against the reference 104.50° , an error of 0.15° . The identity

model, despite having the same 97 parameters, produces the straight line visible in Figure 3 and cannot do otherwise, by the collapse theorem of Section 3.

Where students go wrong: trusting a validation set of five points

The identity model shows a validation RMSE of 549 cm^{-1} against a training RMSE of 2667 cm^{-1} . A validation error five times *smaller* than the training error is not evidence of good generalisation. It happens because the five validation angles fall where the fitted straight line happens to pass close to the curve. With so few points the validation estimate has an enormous variance, and any conclusion drawn from a single small split should be checked by repeating the split.

Chemical connection: reading a force out of the model

Differentiating the trained model gives the restoring torque on the bend,

$$\frac{\partial \hat{E}}{\partial \theta} = \frac{1}{\sigma_\theta} \sum_m w_m g'(z_m) \alpha_m,$$

where the factor $1/\sigma_\theta$ is the chain rule through Equation (37) and is a common source of missing factors. Because g is a Gaussian, g' is smooth and the resulting force is continuous, so the model could be used in a dynamics simulation. Had ReLU units been used, this derivative would be piecewise constant with jumps, and the simulation would not conserve energy.

Part VI. Problems

Solutions to all sixteen problems are given in Appendix B. Attempt each before turning to them.

Problem 1. *Calibration by least squares*

A spectrophotometric calibration gives absorbance A against concentration c in mmol L^{-1} :

c	0.20	0.40	0.60	0.80	1.00
A	0.112	0.219	0.334	0.436	0.548

Using Equation (3), compute the slope and intercept, the residuals, and the sum of squared errors. State what the slope means physically if the path length is 1.00 cm. How many parameters does the model have, and how does that compare with a 1 : 32 : 1 neural model?

Problem 2. *The collapse theorem*

Prove that a 2 : 5 : 5 : 1 model with the identity activation everywhere is equivalent to a single affine map, and state how many independent parameters the equivalent map has. Then explain why the original model still has 71 parameters, and what that discrepancy means for training.

Problem 3. *Parameter counting*

Use Equation (10) to count the parameters of (a) 1 : 32 : 1, (b) 8 : 64 : 64 : 1, (c) 384 : 128 : 128 : 128 : 1. For (c), compare the count with the 2000 reference calculations a typical study can afford, and comment on what must be done to make training viable.

Problem 4. *The logistic function as a population*

(a) Derive $\sigma' = \sigma(1 - \sigma)$ and find the value of v at which the slope is greatest. (b) Show that the population of the upper of two states separated by ΔE equals $\sigma(-\Delta E/k_B T)$. (c) Compute the

population at 300 K for $\Delta E = 600 \text{ cm}^{-1}$, given $k_B T = 208.5 \text{ cm}^{-1}$. (d) In the neuron analogy, what physical quantity does the weight α play the role of?

Problem 5. *Softplus and the partition function*

For a two-level system with the upper level at reduced energy $-v$, write down Z and show that $\ln Z$ is the softplus function. Show that $d(\ln Z)/dv$ is the logistic function and interpret the result thermodynamically. What is the second derivative, and what fluctuation quantity does it correspond to?

Problem 6. *Building a bump*

Using the logistic function, construct a two-neuron model that is close to 1 on $-1 < x < 1$ and close to 0 outside, with edges of width about 0.2. Give α_1, a_1, w_1 and α_2, a_2, w_2 explicitly. How many neurons are needed for n such bumps, and what does the construction say about the number of neurons required to fit a curve with n distinct features?

Problem 7. *Back-propagation by hand*

A 1 : 2 : 1 model with tanh hidden activation has $\alpha = (1.0, -1.5)$, $a = (0.2, -0.4)$, $w = (0.5, 0.9)$ and $b = -0.1$. For the training point $x = 0.6$, $y = 0.30$: (a) compute z , h , $\hat{y}$ and L ; (b) compute all six gradients; (c) take one step with $\gamma = 0.2$ and recompute the loss; (d) which neuron receives the larger correction, and why?

Problem 8. *Learning rate and conditioning*

The Hessian of a loss near its minimum has eigenvalues 0.5 and 40. (a) What is the largest stable learning rate? (b) What is the condition number? (c) With the largest stable rate, by what factor does the error along the *soft* direction decrease per step? (d) State the analogous situation in a geometry optimisation and name the technique used there to fix it.

Problem 9. *Smoothness and forces*

A single-neuron model of a bond stretch is $\hat{E}(r) = w g(\alpha r + a)$ with $\alpha = 4 \text{ \AA}^{-1}$, $a = -6$ and $w = 100 \text{ kJ mol}^{-1}$. (a) With $g = \text{ReLU}$, find $\hat{F} = -d\hat{E}/dr$ on either side of $r = 1.5 \text{ \AA}$ and state the size of the discontinuity. (b) With $g = \tanh$, evaluate $\hat{F}$ at $r = 1.5 \text{ \AA}$. (c) Explain what (a) does to a molecular dynamics simulation, and what it does to a vibrational frequency calculated from the second derivative.

Problem 10. *Ridge regression*

Starting from $L = \|X\beta - y\|^2 + \lambda\|\beta\|^2$, derive the solution. If $X^T X$ has eigenvalues 0.5 and 40, compute the condition number before and after adding $\lambda = 0.5$. Explain in one sentence why the same penalty is called weight decay in a neural context.

Problem 11. *Ensemble uncertainty*

Five independently trained models predict a reaction energy, in hartree: -0.4412 , -0.4388 , -0.4455 , -0.4401 , -0.4370 . Compute the mean, the sample standard deviation, and the standard error of the mean. Convert the standard deviation to kJ mol^{-1} using $1 E_h = 2625.5 \text{ kJ mol}^{-1}$. Would you report this prediction as chemically accurate, and what would you do next?

Problem 12. *Augmentation by permutation*

Compute the number of equivalent atom labellings for H_2O , CH_4 , $\text{C}_2\text{H}_6\text{O}$ and C_6H_6 . For benzene, explain why the full set cannot be used and describe a workable procedure. What property would a model need for this augmentation to be unnecessary?

Problem 13. *Vanishing gradients, twice*

(a) A model has ten stacked logistic layers. Using the bound on σ' , give the largest possible ratio between the gradient at the first layer and at the last. (b) A recurrent model has $\rho = 0.7$ per step. Over a 30-step sequence, what is the surviving factor? (c) Name the architectural fix in each case and say in one sentence why it works.

Problem 14. *Recurrent parameter counts*

For $n = 4$ inputs and hidden length $l = 32$, compute the parameter count of a single layer as a plain recurrent unit, a gated recurrent unit and a long short-term memory unit. If the same layer is stacked twice, what is the second layer's input length, and what are the three totals?

Problem 15. *Diagnosing a training run*

A model is trained and the losses recorded:

epoch	50	200	500	1000	2000	4000
training loss	0.42	0.09	0.021	0.004	0.0006	0.00004
validation loss	0.45	0.11	0.035	0.038	0.055	0.081

(a) At which epoch should training have stopped? (b) What is the diagnosis? (c) Give three remedies and say which is cheapest. (d) If instead both losses had plateaued at 0.30, what would the diagnosis be and what would you change?

Problem 16. *Designing the water calculation*

You are asked to build a model of the bending potential of water over 80° to 180° to an accuracy of 50 cm^{-1} . (a) Justify the choice of activation function. (b) Justify the choice of output activation. (c) Explain why the model of Section 14 must not be used to predict the energy at $\theta = 60^\circ$. (d) The reported test RMSE is 33 cm^{-1} on four points. State one reason to be cautious and one experiment that would settle the matter.

A The script

Every number and both figures in this tutorial come from the following script. It trains the model with the gradients derived in Section 9, written out explicitly rather than obtained from an automatic differentiation library, so that the correspondence with the equations is visible line by line.

```

1  """
2  water_bend_nn.py
3  Every number and figure in the neural-network tutorial is produced here.
4  Dependencies: numpy, matplotlib.
5  """
6  import numpy as np
7  import matplotlib
8  matplotlib.use("Agg")
9  import matplotlib.pyplot as plt
10
11  np.set_printoptions(precision=4, suppress=True)
12  DEEP, TEAL, AMBER, BRICK, SLATE = "#1F3A5F", "#0E6E6E", "#9A6212", "#9E2B25", "#5A6773"
13
14  # =====
15  # 1. Reference bending potential of water
16  #   Model curve: quartic in the angular displacement, with the two
17  #   parameters fixed by (i)  $E = 11000 \text{ cm}^{-1}$  at the linear geometry and
18  #   (ii) the linear geometry being a stationary point by symmetry.
19  # =====
20  THETA_E = 104.5 # degrees
21  K_BEND = 35239.0 #  $\text{cm}^{-1} \text{ rad}^{-2}$ , from  $k_{\text{theta}} = 0.70 \text{ mdyn \AA rad}^{-2}$ 
22  E_LIN = 11000.0 #  $\text{cm}^{-1}$ , barrier to linearity
23
24  u_lin = np.deg2rad(180.0 - THETA_E)

```

```

25 # 0.5 k u^2 + c3 u^3 + c4 u^4 = E_lin ; k u + 3 c3 u^2 + 4 c4 u^3 = 0
26 M = np.array([[u_lin**3, u_lin**4],
27               [3*u_lin**2, 4*u_lin**3]])
28 rhs = np.array([E_LIN - 0.5*K_BEND*u_lin**2, -K_BEND*u_lin])
29 C3, C4 = np.linalg.solve(M, rhs)
30
31 def E_ref(theta_deg):
32     u = np.deg2rad(theta_deg - THETA_E)
33     return 0.5*K_BEND*u**2 + C3*u**3 + C4*u**4
34
35 print("=== reference curve ===")
36 print("c3 = %.1f c4 = %.1f cm^-1 rad^-n" % (C3, C4))
37 for t in [80, 90, 104.5, 120, 150, 180]:
38     print(" E(%5.1f deg) = %9.1f cm^-1" % (t, E_ref(t)))
39
40 # =====
41 # 2. Data set: 25 angles split 16 / 5 / 4
42 # =====
43 rng = np.random.default_rng(7)
44 angles = np.linspace(80.0, 180.0, 25)
45 idx = rng.permutation(25)
46 tr_i, va_i, te_i = np.sort(idx[:16]), np.sort(idx[16:21]), np.sort(idx[21:])
47
48 th_tr, th_va, th_te = angles[tr_i], angles[va_i], angles[te_i]
49 E_tr, E_va, E_te = E_ref(th_tr), E_ref(th_va), E_ref(th_te)
50
51 mu, sig = th_tr.mean(), th_tr.std()
52 Ey, Es = E_tr.mean(), E_tr.std()
53 nx = lambda t: (t - mu)/sig
54 ny = lambda e: (e - Ey)/Es
55 dy = lambda z: z*Es + Ey
56
57 print("\n=== data ===")
58 print("training angles :", np.round(th_tr, 2))
59 print("validation      :", np.round(th_va, 2))
60 print("test            :", np.round(th_te, 2))
61 print("mu_theta = %.4f deg sigma_theta = %.4f deg" % (mu, sig))
62 print("normalised input at theta = 100 deg : %.4f" % nx(100.0))
63
64 # =====
65 # 3. A 1 : M : 1 network, written out by hand
66 # hidden: h_m = g(alpha_m * x + a_m)
67 # output: yhat = b + sum_m w_m h_m
68 # =====
69 def init(M_hidden, seed):
70     r = np.random.default_rng(seed)
71     return dict(alpha=r.normal(0, 1.0, M_hidden), a=r.uniform(-2, 2, M_hidden),
72                w=r.normal(0, 0.3, M_hidden), b=0.0)
73
74 def gauss(v): return np.exp(-v**2)
75 def dgauss(v): return -2*v*np.exp(-v**2)
76 def ident(v): return v
77 def dident(v): return np.ones_like(v)
78
79 def forward(p, x, g):
80     z = np.outer(x, p["alpha"]) + p["a"] # (N, M)
81     h = g(z)
82     return z, h, h @ p["w"] + p["b"]
83
84 def loss_grad(p, x, y, g, dg):
85     z, h, yh = forward(p, x, g)
86     e = yh - y # (N,)
87     N = len(x)
88     gb = 2*e.sum()/N
89     gw = 2*(h*e[:, None]).sum(0)/N
90     d = 2*e[:, None]*p["w"]*dg(z)/N # (N, M)
91     ga = d.sum(0)
92     gal = (d*x[:, None]).sum(0)
93     return (e**2).mean(), dict(alpha=gal, a=ga, w=gw, b=gb)
94
95 def train(p, x, y, xv, yv, g, dg, lr=0.05, mom=0.9, epochs=4000):
96     v = {k: np.zeros_like(np.atleast_1d(p[k]).astype(float)) for k in p}
97     hist = []
98     best, best_p = np.inf, None

```

```

99     for ep in range(epochs):
100         L, G = loss_grad(p, x, y, g, dg)
101         for k in p:
102             v[k] = mom*v[k] - lr*np.atleast_1d(G[k])
103             p[k] = p[k] + (v[k] if np.ndim(p[k]) else float(np.asarray(v[k]).reshape(-1)
104             [0]))
105             Lv = ((forward(p, xv, g)[2] - yv)**2).mean()
106             hist.append((L, Lv))
107             if Lv < best:
108                 best, best_p = Lv, {k: (p[k].copy() if np.ndim(p[k]) else p[k]) for k in p}
109         return best_p, np.array(hist), best
110
111 xtr, ytr = nx(th_tr), ny(E_tr)
112 xva, yva = nx(th_va), ny(E_va)
113 xte, yte = nx(th_te), ny(E_te)
114
115 print("\n=== training ===")
116 p_lin, h_lin, _ = train(init(32, 1), xtr, ytr, xva, yva, ident, dident, lr=0.02, epochs
117                        =4000)
118 p_rbf, h_rbf, _ = train(init(32, 2), xtr, ytr, xva, yva, gauss, dgauss, lr=0.05, epochs
119                        =4000)
120
121 def rmse(p, x, E, g):
122     return float(np.sqrt(np.mean((dy(forward(p, x, g)[2]) - E)**2)))
123
124 for name, p, g, h in [("identity activation", p_lin, ident, h_lin),
125                      ("Gaussian RBF", p_rbf, gauss, h_rbf)]:
126     print("%s RMSE train %8.1f val %8.1f test %8.1f cm-1"
127           % (name, rmse(p, xtr, E_tr, g), rmse(p, xva, E_va, g), rmse(p, xte, E_te, g)))
128
129 # predicted equilibrium angle from the RBF model
130 grid = np.linspace(80, 180, 2001)
131 pred = dy(forward(p_rbf, nx(grid), gauss)[2])
132 print("RBF model minimum at theta = %.2f deg (reference %.2f)" % (grid[np.argmin(pred)],
133                        THETA_E))
134
135 # =====
136 # 4. Hand-worked forward and backward pass on a 1 : 2 : 1 tanh network
137 # =====
138 print("\n=== hand-worked 1:2:1 network, tanh hidden, one training point ===")
139 al = np.array([1.5, -2.0]); aa = np.array([0.5, 0.4])
140 ww = np.array([0.8, -0.6]); bb = 0.1
141 x0 = float(nx(100.0)); y0 = float(ny(E_ref(100.0)))
142 print("point: theta = 100 deg, x = %.4f, y = %.4f (normalised)" % (x0, y0))
143 z = al*x0 + aa
144 h = np.tanh(z)
145 yh = float(h @ ww + bb)
146 e = yh - y0
147 L = e**2
148 print("z      =", np.round(z, 6))
149 print("h      =", np.round(h, 6))
150 print("yhat   = %.6f   y = %.6f   e = %.6f   L = %.6f" % (yh, y0, e, L))
151 dLdb = 2*e
152 dLdw = 2*e*h
153 delta = 2*e*ww*(1 - h**2)
154 dLda = delta
155 dLdal = delta*x0
156 print("dL/db   = %.6f" % dLdb)
157 print("dL/dw   =", np.round(dLdw, 6))
158 print("delta   =", np.round(delta, 6))
159 print("dL/da   =", np.round(dLda, 6))
160 print("dL/dal  =", np.round(dLdal, 6))
161 lr = 0.1
162 print("updated w =", np.round(ww - lr*dLdw, 6), " b = %.6f" % (bb - lr*dLdb))
163 zn = (al - lr*dLdal)*x0 + (aa - lr*dLda)
164 yn = float(np.tanh(zn) @ (ww - lr*dLdw) + (bb - lr*dLdb))
165 print("new yhat = %.6f   new L = %.6f   (was %.6f)" % (yn, (yn - y0)**2, L))
166
167 # =====
168 # 5. Miscellaneous numbers quoted in the problems
169 # =====
170 print("\n=== sigmoid as a two-state Boltzmann population ===")
171 kT300 = 0.695034*300.0 # cm-1 at 300 K
172 for dE in [0.0, 200.0, 600.0, 1500.0]:

```

```

169     print(" dE = %6.1f cm-1 : p_upper = %.4f" % (dE, 1/(1+np.exp(dE/kT300))))
170     print(" kT at 300 K = %.1f cm-1" % kT300)
171
172     print("\n=== bump from two sigmoids ===")
173     s = lambda v: 1/(1+np.exp(-v))
174     for v in [-2, -1, 0, 1, 2]:
175         print(" v = %d : sigma(10(v+1)) - sigma(10(v-1)) = %.5f" % (v, s(10*(v+1))-s(10*(v-1))))
176
177     print("\n=== parameter counts ===")
178     for (p_in, M_, out) in [(1, 32, 1), (1, 64, 1), (3, 64, 64), (384, 128, 1)]:
179         print(" %d:%d:%d -> %d parameters" % (p_in, M_, out, (p_in+1)*M_ + (M_+1)*out))
180     l, n = 25, 3
181     print(" RNN layer 1 l=25 n=3 : %d ; layer 2 : %d" % (1*(l+n+1), 1*(l+l+1)))
182     print(" LSTM equivalents : %d and %d" % (4*1*(l+n+1), 4*1*(l+l+1)))
183     print(" GRU equivalents : %d and %d" % (3*1*(l+n+1), 3*1*(l+l+1)))
184
185     print("\n=== vanishing gradient ===")
186     for w in [0.5, 0.9, 1.1]:
187         print(" |w| = %.1f over 20 steps : factor %.3e" % (w, w**20))
188
189     print("\n=== ensemble uncertainty ===")
190     ens = np.array([-76.4312, -76.4298, -76.4325, -76.4301, -76.4288, -76.4319, -76.4307, -76.4295])
191     print(" mean = %.5f Eh sd = %.2e Eh sem = %.2e Eh"
192           % (ens.mean(), ens.std(ddof=1), ens.std(ddof=1)/np.sqrt(len(ens))))
193     print(" sd in kJ/mol = %.3f" % (ens.std(ddof=1)*2625.5))
194
195     print("\n=== augmentation by permutation of identical atoms ===")
196     from math import factorial
197     for name, counts in [("H2O", [1, 2]), ("CH4", [1, 4]), ("C6H6", [6, 6]), ("C2H6O", [2, 6, 1])]:
198         n_perm = np.prod([factorial(c) for c in counts])
199         print(" %-6s : %d equivalent labellings" % (name, n_perm))
200
201     print("\n=== learning-rate stability on a quadratic ===")
202     for k in [1.0, 10.0]:
203         print(" curvature k = %4.1f : stable for gamma < %.3f" % (k, 2/k))
204     for g_ in [0.05, 0.19, 0.21]:
205         x_ = 1.0
206         for _ in range(30):
207             x_ = x_ - g_*(10.0*x_)
208         print(" k=10, gamma=%.2f : x after 30 steps = %.4g" % (g_, x_))
209
210     # =====
211     # 6. Figures
212     # =====
213     fig, ax = plt.subplots(1, 2, figsize=(10.4, 4.0))
214     tt = np.linspace(80, 180, 400)
215     ax[0].plot(tt, E_ref(tt), color=SLATE, lw=1.2, label="reference curve")
216     ax[0].plot(tt, dy(forward(p_lin, nx(tt), ident)[2]), color=BRICK, lw=1.6, ls="--",
217                  label="identity activation")
218     ax[0].plot(tt, dy(forward(p_rbf, nx(tt), gauss)[2]), color=TEAL, lw=1.8,
219                  label="Gaussian RBF, 1:32:1")
220     ax[0].scatter(th_tr, E_tr, s=26, color=DEEP, zorder=5, label="training (16)")
221     ax[0].scatter(th_va, E_va, s=34, facecolor="white", edgecolor=AMBER, lw=1.4, zorder=5,
222                  label="validation (5)")
223     ax[0].scatter(th_te, E_te, s=40, marker="s", facecolor="white", edgecolor=BRICK, lw=1.4,
224                  zorder=5, label="test (4)")
225     ax[0].axvline(THETA_E, color=SLATE, lw=0.6, ls=":")
226     ax[0].set_xlabel(r"$H$-$O$-$H$ angle $\theta$ / degrees")
227     ax[0].set_ylabel(r"$E$ / cm-1")
228     ax[0].set_title("Bending potential of water", fontsize=10, color=DEEP)
229     ax[0].legend(fontsize=7, frameon=False)
230
231     # --- overfitting demonstration: 8 noisy training points, long training ---
232     rng2 = np.random.default_rng(11)
233     sub = np.array([0, 2, 4, 7, 9, 11, 13, 15])
234     xof = xtr[sub]
235     yof = ytr[sub] + rng2.normal(0, 0.04, len(sub)) # 4% noise on normalised energy
236     p_of, h_of, best_of = train(init(32, 5), xof, yof, xva, yva, gauss, dgauss,
237                                lr=0.05, epochs=3000)
238     kbest = int(np.argmin(h_of[:, 1]))
239     print("\n=== overfitting demonstration ===")

```

```

240 print("8 noisy training points, 32 hidden neurons, 3000 epochs")
241 print("minimum validation loss %.5f at epoch %d" % (h_of[kbest, 1], kbest+1))
242 print("validation loss at the final epoch %.5f" % h_of[-1, 1])
243 print("training loss at the final epoch %.5f" % h_of[-1, 0])
244 print("noise level added to targets: sd %.1f cm-1" % (0.04*Es))
245
246 ep = np.arange(1, len(h_of)+1)
247 ax[1].semilogy(ep, h_of[:, 0], color=DEEP, lw=1.2, label="training loss")
248 ax[1].semilogy(ep, h_of[:, 1], color=AMBER, lw=1.2, label="validation loss")
249 ax[1].axvline(kbest+1, color=BRICK, lw=0.9, ls="--")
250 ax[1].annotate("early stopping\nePOCH %d" % (kbest+1), xy=(kbest+1, h_of[kbest, 1]),
251               xytext=((kbest+1)*0.06, h_of[kbest, 1]*25), fontsize=7, color=BRICK,
252               arrowprops=dict(arrowstyle=">", color=BRICK, lw=0.8))
253 ax[1].set_xlabel("epoch")
254 ax[1].set_ylabel("mean squared error (normalised units)")
255 ax[1].set_ylim(1e-8, 3)
256 ax[1].set_title("Overfitting: 8 noisy training points", fontsize=10, color=DEEP)
257 ax[1].legend(fontsize=7, frameon=False)
258
259 for a in ax:
260     a.spines["top"].set_visible(False); a.spines["right"].set_visible(False)
261     a.tick_params(labelsize=8)
262 plt.tight_layout()
263 plt.savefig("fig_water.png", dpi=220)
264 print("\nwrote fig_water.png")
265
266 # activation functions and their derivatives
267 fig2, ax2 = plt.subplots(1, 2, figsize=(10.4, 3.4))
268 v = np.linspace(-4, 4, 600)
269 funcs = [("ReLU", np.maximum(v, 0), np.where(v > 0, 1.0, 0.0), BRICK),
270          ("logistic", s(v), s(v)*(1-s(v)), DEEP),
271          ("tanh", np.tanh(v), 1-np.tanh(v)**2, TEAL),
272          ("Gaussian RBF", np.exp(-v**2), -2*v*np.exp(-v**2), AMBER)]
273 for nm, f, df, c in funcs:
274     ax2[0].plot(v, f, color=c, lw=1.5, label=nm)
275     ax2[1].plot(v, df, color=c, lw=1.5, label=nm)
276 ax2[0].set_title(r"$g(v)$", fontsize=10, color=DEEP)
277 ax2[1].set_title(r"$\frac{d}{dv}g$", fontsize=10, color=DEEP)
278 for a in ax2:
279     a.axhline(0, color=SLATE, lw=0.4); a.axvline(0, color=SLATE, lw=0.4)
280     a.spines["top"].set_visible(False); a.spines["right"].set_visible(False)
281     a.set_xlabel("$v$"); a.tick_params(labelsize=8); a.legend(fontsize=7, frameon=False)
282 plt.tight_layout()
283 plt.savefig("fig_activations.png", dpi=220)
284 print("wrote fig_activations.png")

```

Listing 1: *water_bend_nn.py*. Reference curve, data split, forward and backward passes, training, the hand-worked example, and all auxiliary numbers.

B Solutions

1. $\bar{c} = 0.600$, $\bar{A} = 0.3298$. The sums give $w = 0.5445 \text{ L mmol}^{-1}$ and $b = 0.0031$. Residuals are $(0.0000, -0.0019, +0.0042, -0.0027, +0.0004)$ with $\text{SSE} = 2.87 \times 10^{-5}$ and $R^2 = 0.99976$. With $\ell = 1.00 \text{ cm}$ the slope is the molar absorption coefficient, $\varepsilon = 545 \text{ L mol}^{-1} \text{ cm}^{-1}$ after converting the concentration unit. The model has two parameters against 97 for the neural model, a ratio of nearly fifty, which is the price of not knowing the functional form in advance.

2. Composing $h^{(1)} = W^{(1)}x + b^{(1)}$, $h^{(2)} = W^{(2)}h^{(1)} + b^{(2)}$ and $\hat{y} = w^{(3)\top}h^{(2)} + b^{(3)}$ gives $\hat{y} = w'^{\top}x + b'$ with $w'^{\top} = w^{(3)\top}W^{(2)}W^{(1)}$. The equivalent map has three independent parameters, two weights and one bias. The original model has $3 \cdot 5 + 6 \cdot 5 + 6 \cdot 1 = 71$ parameters, so 68 directions in parameter space leave the function unchanged. Training on such a surface is degenerate: the minimum is a 68-dimensional flat manifold rather than a point, the Hessian is singular, and the extra parameters add cost without capability.

3. (a) $(1+1)32 + (32+1)1 = 97$. (b) $(8+1)64 + (64+1)64 + (64+1)1 = 576 + 4160 + 65 = 4801$. (c) $(384+1)128 + (128+1)128 \times 2 + (128+1)1 = 49\,280 + 33\,024 + 129 = 82\,433$. With 2000 reference points there are forty parameters per data point, so unregularised training will interpolate the training set exactly. Viable training requires some combination of augmentation, heavy regularisation, early

stopping, a smaller architecture, and training on forces as well as energies, which multiplies the number of constraints by $3N_{\text{atoms}}$ per structure.

4. (a) Given in Section 5; the slope is greatest at $v = 0$ where $\sigma' = \frac{1}{4}$, since $\sigma(1 - \sigma)$ is maximised at $\sigma = \frac{1}{2}$. (b) $p_{\text{upper}} = e^{-\Delta E/k_B T} / (1 + e^{-\Delta E/k_B T}) = 1 / (1 + e^{\Delta E/k_B T}) = \sigma(-\Delta E/k_B T)$. (c) $\sigma(-600/208.5) = \sigma(-2.878) = 0.0533$, so about 5.3%. (d) The weight is an inverse temperature. A large $|\alpha|$ makes the neuron a sharp switch, corresponding to a low temperature at which the population changes abruptly across the gap.

5. $Z = 1 + e^v$ and $\ln Z = \ln(1 + e^v)$, which is the softplus function. Differentiating, $d \ln Z / dv = e^v / (1 + e^v) = \sigma(v)$, the population of the upper level, which is the usual statement that the derivative of $\ln Z$ with respect to a reduced energy returns an average occupation. The second derivative is $\sigma(1 - \sigma)$, which is the variance of the occupation, and in thermodynamic language it is the fluctuation quantity that gives the heat capacity.

6. Take $\alpha_1 = 10$, $a_1 = +10$, $w_1 = +1$ and $\alpha_2 = 10$, $a_2 = -10$, $w_2 = -1$, that is $B(x) = \sigma(10(x + 1)) - \sigma(10(x - 1))$. The edges have width of order $4/\alpha = 0.4$ measured between the 10% and 90% points, and the tabulated values in Section 4 confirm $B(0) = 0.99991$ and $B(\pm 2) = 0.00005$. Each bump needs two neurons, so n bumps need $2n$. The practical reading is that the number of hidden neurons should scale with the number of distinct features in the target curve, not with the number of data points.

7. (a) $\mathbf{z} = (0.800, -1.300)$, $\mathbf{h} = (0.6640, -0.8617)$, $\hat{y} = -0.5435$, $e = -0.8435$, $L = 0.7116$. (b) $\partial L / \partial b = 2e = -1.6871$; $\partial L / \partial w = 2eh = (-1.1203, +1.4538)$; $\mathbf{g}'(\mathbf{z}) = (0.5591, 0.2574)$; $\delta = 2ew \odot \mathbf{g}' = (-0.4716, -0.3909)$, which is also $\partial L / \partial \mathbf{a}$; and $\partial L / \partial \mathbf{a} = \delta \mathbf{x} = (-0.2829, -0.2345)$. (c) After one step with $\gamma = 0.2$, $\hat{y} = 0.2591$ and $L = 0.00167$, a reduction by a factor of 425. The step is large because the initial error is large and the loss is locally close to quadratic. (d) Neuron 1 receives the larger correction. Its gradient carries $\mathbf{g}' = 0.5591$ against 0.2574 for neuron 2, which is closer to saturation at $|h| = 0.86$. The smaller output weight of neuron 1 partly offsets this, but the activation factor dominates.

8. (a) $\gamma_{\text{max}} = 2/\lambda_{\text{max}} = 2/40 = 0.05$. (b) $\kappa = 40/0.5 = 80$. (c) Along the soft direction the contraction per step is $|1 - \gamma\lambda_{\text{min}}| = |1 - 0.05 \times 0.5| = 0.975$, so the error falls by only 2.5% per step while the stiff direction is already converged. Roughly κ steps are needed, which is the general result. (d) The same situation arises in a geometry optimisation of a molecule with stiff bond stretches and a floppy torsion. The fix is to work in coordinates that decouple the modes, or to use a quasi-Newton method that builds an approximate inverse Hessian, which is the optimisation analogue of the adaptive learning-rate methods used in training.

9. At $r = 1.5 \text{ \AA}$ the pre-activation is $z = 4(1.5) - 6 = 0$, exactly the kink. (a) For ReLU, $\hat{E} = 0$ for $r < 1.5$ so $\hat{F} = 0$, and $\hat{E} = 100(4r - 6)$ for $r > 1.5$ so $\hat{F} = -400 \text{ kJ mol}^{-1} \text{ \AA}^{-1}$. The force jumps by $400 \text{ kJ mol}^{-1} \text{ \AA}^{-1}$ across a single point. (b) For tanh, $\hat{F} = -w\alpha \mathbf{g}'(z) = -100 \times 4 \times (1 - \tanh^2 0) = -400 \text{ kJ mol}^{-1} \text{ \AA}^{-1}$, the same value but approached continuously from both sides. (c) In dynamics the discontinuity delivers an impulsive kick whenever a trajectory crosses the threshold, so energy is not conserved and the integration timestep must be reduced without limit. For frequencies the second derivative of a ReLU model is zero almost everywhere and undefined at the kink, so a harmonic analysis returns either zero or nonsense.

10. $\partial L / \partial \boldsymbol{\beta} = 2\mathbf{X}^\top (\mathbf{X}\boldsymbol{\beta} - \mathbf{y}) + 2\lambda\boldsymbol{\beta} = \mathbf{0}$ gives $\boldsymbol{\beta} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$. Every eigenvalue is shifted by λ , so the condition number falls from $40/0.5 = 80$ to $40.5/1.0 = 40.5$. In a neural context the gradient of the penalty is $2\lambda\boldsymbol{\theta}$, so each update subtracts a term proportional to the parameter itself, which shrinks every parameter geometrically towards zero in the absence of data pressure, hence weight decay.

11. Mean -0.44052 E_h , sample standard deviation $3.19 \times 10^{-3} \text{ E}_h$, standard error $1.43 \times 10^{-3} \text{ E}_h$. The spread is 8.4 kJ mol^{-1} , which is well outside chemical accuracy of about 4 kJ mol^{-1} , so the prediction should not be reported as chemically accurate. The large spread indicates that the models disagree, which usually means the input is outside the region covered by the training data. The next step is to check whether the structure lies inside the training distribution and, if it does not, to compute a reference value for it and retrain.

12. $1!2! = 2$ for water, $1!4! = 24$ for methane, $2!6!1! = 1440$ for ethanol, and $6!6! = 518400$ for benzene. The benzene set is far larger than any training set and enumerating it would swamp the data with copies of a handful of structures, so a small random sample of permutations is drawn afresh each epoch. Augmentation is unnecessary if the model is permutation invariant by construction, which is precisely what a message-passing architecture on the molecular graph provides.

13. (a) $\sigma' \leq \frac{1}{4}$, so ten layers give at most $(1/4)^{10} = 9.5 \times 10^{-7}$. (b) $0.7^{30} = 2.3 \times 10^{-5}$. (c) For the deep feed-forward case, residual connections add an identity path so that the derivative contains a term equal to one and the product no longer decays. For the recurrent case, gating with a forget gate near one makes the cell state propagate almost unchanged, achieving the same effect along the time axis.

14. $l(l + n + 1) = 32(32 + 4 + 1) = 32 \times 37 = 1184$ for the plain unit, 3552 for the gated recurrent unit and 4736 for the long short-term memory unit. The second layer receives the hidden state of the first, so its input length is $l = 32$ and its count is $32(32 + 32 + 1) = 2080, 6240$ and 8320 respectively. Two-layer totals are 3264, 9792 and 13 056.

15. (a) At epoch 500, where the validation loss reaches its minimum of 0.035. (b) Overfitting: the training loss falls by three further orders of magnitude while the validation loss more than doubles. (c) Early stopping with a patience of a few hundred epochs, which costs nothing and is the cheapest; weight decay with λ tuned on the validation set; more data or augmented data; a smaller architecture; dropout. (d) Two flat losses at a high value is underfitting, and the response is the opposite: a larger model, a more expressive activation function, longer training, or better input features. Checking that the activation function is not the identity is the first thing to do.

16. (a) A smooth activation function is required, because the derivative of the model with respect to the angle is the restoring torque and must be continuous. A Gaussian is a good choice for a one-dimensional curve because its locality suits a well-shaped target, and GELU would also serve. ReLU is excluded by Problem 9. (b) The output activation must be the identity, since the energy is an unbounded real number and any saturating function would cap the predictable range and flatten the gradient at large energies. (c) At $\theta = 60^\circ$ the model extrapolates outside the training range $[80^\circ, 180^\circ]$. Gaussian units are local, so every hidden unit returns nearly zero there and the prediction decays to the output bias b , which is a meaningless number. Extrapolation failure is silent, since the model reports no warning. (d) Four test points give a very uncertain error estimate, and the training and test RMSE differ by a factor of 1.7 which is within the noise of such a small sample. The experiment that settles it is to repeat the whole procedure over several random splits and several random initialisations, and to report the mean and spread of the test RMSE rather than a single value.

End of tutorial